

Comparative Evaluation of KV Cache Optimization Strategies for Efficient LLM Inference: Quantization, Sparse Attention, Eviction, and Latent Projection

Rishika Mamidibathula[†], Suhas Morisetty^{*}, Ziheng Wang^{*}, Tony Tian^{*}

^{*}Department of Computer Science, [†]Department of Data Science, Columbia University
COMS E6998: High Performance Machine Learning, Spring 2026
{rm4318, vm2825, zw3153, jt3640}@columbia.edu

Abstract—Autoregressive large language model (LLM) decoding is bounded by key-value (KV) cache memory bandwidth: at batch size 32 and 4K context, Llama-2-7B requires ≈ 65 GB for KV alone, nearly saturating an H100 80GB GPU. We present a single-codebase comparison of five KV-cache optimization techniques spanning four compression families (quantization via KIVI 2/4-bit, sparse dynamic selection via TopK / TokenSelect, one-shot eviction via SnapKV, and latent projection via TransMLA), plus a quantized-sparse hybrid (KIVI $\times$ TopK). All methods share the same model weights, tokenizer, greedy decode loop, prompts, seeds, and GPU (NVIDIA H100 80GB HBM3) so that differences are attributable to the KV policy only. Our main results: KIVI 4-bit incurs no measurable LongBench quality drop (0.292 vs. 0.291) while delivering a $2.3\times$ smaller KV cache and $1.93\times$ decode throughput at BS=32, with maximum single-GPU throughput rising $3.1\times$ (1,517 vs. 497 tok/s) once the larger batch ceiling is exploited. SnapKV marginally improves overall quality (+0.3%) with zero per-step overhead. TopK K=1024 matches baseline quality and gains +20% on TriviaQA by suppressing distractor passages. MLA achieves a consistent $3.9\times$ KV reduction and $1.3\times$ decode speedup at batch size 1, but the available checkpoint exhibits a large quality regression that requires retraining at longer context. No single method is best on all three metrics, and the right choice depends on whether batch size, context length, or output fidelity is the binding constraint.

Index Terms—KV cache, LLM inference, quantization, sparse attention, token eviction, multi-head latent attention, throughput, memory efficiency

I. Introduction

Large language model inference operates in two phases: prefill, which processes the entire input prompt in a single forward pass, and decode, which generates one token per step. During decode, all past key and value tensors must be read from GPU HBM at every step, making decode strongly memory-bandwidth bound. For Llama-2-7B [2] with 32 transformer layers, 32 attention heads of dimension 128, FP16 precision, 4096-token context, and batch size 32, the KV cache alone occupies

$2 \times 32 \times 32 \times 128 \times 4096 \times 32 \times 2 \approx 65$ GB —nearly the full capacity of an H100 80 GB GPU [1].

Research on KV cache reduction has produced four principal compression families: (1) Quantization—reduce bits per element while keeping all tokens [3], [4]; (2) Sparse selection—attend to a dynamically chosen subset of tokens per step [5], [6]; (3) Eviction—permanently discard low-importance tokens after prefill [8], [9], [10]; (4) Latent projection—store a compressed low-rank representation instead of full K/V tensors [11], [12].

Published evaluations rarely share identical experimental conditions, making cross-family comparisons unreliable. This work addresses that gap with a unified benchmark harness that holds model, hardware, decode loop, prompts, and seeds constant across all methods.

Our main contributions are: (i) a single-codebase benchmark of five KV optimization techniques including a quantized-sparse hybrid; (ii) kernel-level analysis explaining crossover behavior (KIVI and TopK are slower than baseline at low batch sizes but faster at high batch sizes or long context); (iii) empirical evidence that KIVI 4-bit and SnapKV 0.4 provide lossless or better LongBench quality; (iv) full reproducibility via Modal cloud scripts and a public W&B dashboard at <https://wandb.ai/rm4318-columbia-university/kv-cache-hpml>.

II. Literature Review

KV Quantization. KIVI [3] stores keys with per-channel scales (capturing the dominant channel-direction outliers) and values with per-token grouped quantization (group size 32), retaining the last 128 tokens in FP16 to preserve short-range attention fidelity. At 2-bit it achieves roughly $8\times$ cache compression; at 4-bit, $4\times$. KVQuant [4] extends this with per-channel smoothing and non-uniform codebooks, reaching near-lossless 1-bit compression at the cost of greater overhead.

Sparse Token Selection. TokenSelect [5] (the basis of our TopK implementation) maintains the full FP16 cache but at each decode step selects only the top- K tokens for attention, fusing the scoring kernel in Triton. A cosine-similarity reuse cache avoids re-scoring when consecutive queries are similar. RocketKV [6] adds a two-stage compression pipeline; RefreshKV [7] periodically refreshes evicted tokens.

Eviction-Based Methods. H2O [8] identifies “heavy hitter” tokens by cumulative attention mass and evicts the rest. SnapKV [9] performs a single eviction after prefill by voting over a small observation window of recent queries, producing a fixed-size compressed cache with no per-step overhead. Ada-KV [10] allocates the budget adaptively per head.

Latent KV / Architectural Compression. DeepSeek-V2 [11] introduced Multi-head Latent Attention (MLA), which stores a compressed latent vector c_t per token and expands to full K/V during attention via a learned up-projection, using Attention Absorption to amortize the projection cost. TransMLA [12] retrofits this structure onto an existing Llama-2 checkpoint.

III. Methodology

A. Model and Hardware

All experiments use meta-llama/Llama-2-7b-chat-hf (32 layers, 32 heads, head dim 128, FP16 weights) on a single NVIDIA H100 80GB HBM3 GPU provided by Modal cloud compute. Software stack: PyTorch 2.4.1, HuggingFace Transformers 4.44.2, CUDA 12.1, Triton 3.0, flash-attn 2.5. The MLA method additionally uses botsxc/llama2-7b-chat-mla-2048-8 [12] (freqfold-8, 2048-dim latent, calibrated at 256-token context, no finetuning).

B. Evaluation Datasets and Metrics

LongBench quality [14]: 6 long-context tasks $\times$ 20 examples = 120 examples per method. Tasks: qasper (scientific QA, F1), multifieldqa_en (multi-document QA, F1), triviaqa (open-domain QA, F1), 2wikimqa (multi-hop QA, F1), multi_news (summarization, ROUGE-L), lcc (code completion, edit similarity). Input truncated to 4096 tokens using middle truncation (KIVI paper convention).

Throughput benchmarks: synthetic fixed-size batches; prefill length 1024 tokens, generation 512 tokens, batch sizes 1–256. Five batches per config; first excluded as warmup; mean of four reported.

Long-context benchmarks: prefill length 32,768 tokens, generation 512 tokens, batch size 1, using TopK-Flash and the KIVI $\times$ TopK hybrid.

C. Profiling Tools

Memory: torch.cuda.max_memory_allocated() reset before each batch; measures peak, not steady-state. Timing: torch.cuda.synchronize() + time.perf_counter() for TTFT (time to first token, i.e., prefill) and decode throughput (tokens per second, steps 2– G). Experiment

tracking: Weights & Biases; all runs at <https://wandb.ai/rm4318-columbia-university/kv-cache-hpml>. Kernel profiling: torch.profiler with Chrome trace export for operator-level analysis.

D. Method Implementations

All methods implement a shared MethodWrapper interface in methods/ and are invoked through an identical generate loop in benchmark/runner.py.

Baseline (FP16): Standard dense attention with no KV modification; serves as the reference.

KIVI [3]: Keys quantized per-channel to 2 or 4 bits (grouping across the sequence dimension per head); values quantized per-token with group size 32. The last residual_length=128 tokens are retained in FP16. Quantization uses a custom Triton kernel (triton_quantize_and_pack) that fuses INT4 packing; GEMV dequantizes on-the-fly via cuda_bmm_fA_qB_outer. The KV cache is stored as a 9-tuple (quantized blocks + scales + zeros per K and V + FP16 residual buffer) per layer.

TopK / TokenSelect [5]: The full FP16 cache is retained. Three-region attention budget: sink window (first 128 tokens), dynamically-selected top- K middle tokens, and local window (last 512 tokens). Per-decode-step scoring fuses $\mathbf{QK}^\top$ across all 32 heads via Triton kernel _topk_score_kernel $\rightarrow$ softmax $\rightarrow$ per-head sum $\rightarrow$ torch.topk. A cosine similarity cache (threshold 0.9) skips re-scoring when consecutive queries are similar. RoPE correction passes true_seq_length to each decode step.

TopK-Flash: Variant of TopK using a paged KV pool (256-token pages) + flash_attn_with_kvcache for zero-copy selective attention. Avoids the 52GB contiguous allocation required by standard flash-attn at 32K context.

SnapKV [9]: After prefill, averages attention scores over the last 32 query positions per head (the observation window), applies max-pool (kernel size 7) to smooth noise, then retains $\lfloor S \times 0.4 \rfloor$ top-voted tokens per head permanently. Decode runs over the fixed reduced cache with zero per-step overhead. Budget ratio 0.4; observation window 32.

MLA (TransMLA [12]): Stores a compressed latent vector $c_t \in \mathbb{R}^{d_c}$ per token (here $d_c = 2048$, $4\times$ compression vs. $d_k \cdot n_h = 4096$). At decode time, c_t is up-projected to full K/V by learned matrices; Attention Absorption folds the up-projection into QKV attention.

KIVI $\times$ TopK Hybrid: Combines KIVI (INT4 packed storage) with sparse attention: prefill uses F.scaled_dot_product_attention (flash-attn backed, $O(T)$ memory) to avoid 32K OOM; decode scores all tokens via cuda_bmm_fA_qB_outer over INT4-packed K, selects $n_{\text{sink}} + K + n_{\text{local}}$ positions, then gathers packed INT4 V rows directly for the attention GEMV without an intermediate FP16 write.

TABLE I

LongBench quality (F1 / ROUGE-L / edit-sim), 6 tasks $\times$ 20 ex.
Tasks: (1) qasper, (2) mfqa, (3) triviaqa, (4) 2wikimqa, (5) mnews,
(6) lcc.

Method	(1)	(2)	(3)	(4)	(5)	(6)	Avg
Baseline (FP16)	.172	.380	.644	.083	.181	.287	.291
KIVI 4-bit	.194	.360	.643	.083	.188	.286	.292
KIVI 2-bit	.133	.382	.608	.050	.191	.277	.273
TopK K=2048	.145	.329	.731	.075	.178	.282	.290
TopK K=1024	.153	.281	.773	.089	.183	.273	.292
TopK K=512	.145	.237	.751	.050	.178	.281	.274
SnapKV 0.4	.153	.387	.687	.083	.181	.277	.295
MLA (B=1)*	.036	.101	.064	.043	.103	.140	.081

baseline truncated to 2048 tokens vs. 4096 for others.

TABLE II

Decode throughput (tok/s) and peak GPU memory vs. batch size.
Prefill 1024, gen 512. “—” = OOM.

BS	Decode tok/s			Peak mem (GB)		
	Base	KIVI-2b	Spdup	Base	KIVI-2b	MLA
1	70	30	0.43 $\times$	15.1	14.0	13.8
2	135	61	0.45 $\times$	16.7	14.4	—
4	242	123	0.51 $\times$	20.0	15.3	—
8	351	243	0.69 $\times$	26.4	17.1	—
16	438	480	1.10 $\times$	39.3	20.7	—
32	497	957	1.93 $\times$	65.0	27.8	—
64	—	1295	∞	—	42.1	—
128	—	1517	∞	—	70.7	—

IV. Experimental Results

A. LongBench Quality

Table I reports per-task and overall LongBench scores for all methods. KIVI 4-bit and TopK K=1024 both achieve 0.292 (+0.3% and +0.4% over baseline 0.291) — within evaluation noise for 20 examples per task but consistent across runs. SnapKV 0.4 reaches 0.295 (+0.3%).

Per-task patterns show two opposite breakdown mechanisms: KIVI 2-bit and TopK K=512 both converge to overall score ≈ 0.274 via different paths. KIVI’s quantization noise most hurts qasper (−22%) and 2wikimqa (−40%), tasks requiring precise recall of sparse mid-context facts. TopK’s sparse selection most hurts multifieldqa_en (−38% at K=512) because answer spans at depth $> n_{\text{sink}} + n_{\text{local}}$ are pruned. Conversely, TopK beats baseline on TriviaQA across all K (+13% to +20%): the answer appears in multiple phrasings; sparse selection drops distracting copies and focuses attention. SnapKV’s one-shot vote actively aligns selection with answer-bearing positions, boosting triviaqa (+4%) and multifieldqa (+0.7%) while only hurting qasper (−2%) where evidence spans fall outside the 32-token observation window.

B. Throughput and Memory

Table II shows the KIVI throughput sweep at prefill 1024, gen 512.

Crossover at BS=16. KIVI 2-bit decode throughput is 0.4–0.7 $\times$ baseline at BS 1–8 but crosses over at BS=16

TABLE III

Long-context results. Prefill 32,768 tokens, gen 512, BS=1.

Method	TTFT (ms)	Spdup	Peak mem	Saving
Baseline (FP16)	2627	1.00 $\times$	52.1 GB	—
TopK-Flash	1761	1.49 $\times$	42.1 GB	−19%
SnapKV 0.4	—	—	26.0 GB [†]	−50%
KIVI $\times$ TopK 4b	—	0.60 $\times$	25.8 GB	−51%

Baseline OOMs at 64K; TopK-Flash succeeds at 65.8 GB. [†]SnapKV peak memory at 32K is interpolated from short-context measurements; 32K SnapKV throughput not directly benchmarked.

(480 vs. 438 tok/s, 1.10 $\times$) and reaches 1.93 $\times$ at BS=32. The crossover arises because at small batch the decode $\mathbf{QK}^\top$ is a GEMV: INT4 GEMV does not map to tensor core units and is $\approx 2.5\times$ slower than FP16 GEMV. At large batch the operation becomes a near-GEMM where HBM bandwidth saving (the KV cache is 2.3 $\times$ smaller at BS=32: 27.8 vs. 65 GB) outweighs dequantization cost.

Capacity extension. Baseline OOMs at BS=64; KIVI 2-bit reaches BS=128 at 70.7 GB and achieves 1,517 tok/s—3.1 $\times$ the maximum ever achievable with the baseline on this GPU.

TTFT overhead. KIVI TTFT is 18–66% worse across batch sizes (e.g., 998 vs. 1659 ms at BS=32) because the full KV cache is quantized and packed at the end of prefill—a cost proportional to $B \times S$. For latency-sensitive workloads (interactive chat), this is a real penalty; for throughput-oriented offline inference, KIVI is the preferred choice.

MLA at B=1. MLA achieves 43–44 tok/s vs. 34 tok/s baseline (1.26–1.32 $\times$) despite batch size 1. This confirms that even single-request decode is partially HBM-bandwidth bound; the 3.9 $\times$ KV reduction reduces per-step HBM traffic proportionally. TTFT is also lower with MLA at shorter prefill lengths (39 vs. 52 ms at $p=512$) due to the smaller working set.

C. Long-Context Results (32K)

TopK-Flash. At 32K context the 256-token paged KV pool eliminates the 52 GB contiguous memory allocation that standard flash-attn requires, reducing TTFT by 33% and peak memory by 19%. The gain is not from sparsity per se but from allocation and memory-access-pattern improvement. Baseline OOMs at 64K; TopK-Flash handles it at 65.8 GB.

KIVI $\times$ TopK hybrid. At 32K, combining INT4 storage with sparse attention cuts peak memory 51% (52.1 $\rightarrow$ 25.8 GB) but decode speed drops 40% (21.4 $\rightarrow$ 12.9 tok/s). The slowdown arises because $\mathbf{QK}^\top$ must still score all 32K tokens every step before top- K can be applied—the scoring cost is identical to the baseline, plus additional topk + sort + gather overhead. LongBench quality remains at ≈ 0.297 (within noise of 0.291 baseline).

TABLE IV
Deployment fit summary.

Method	KV ratio	Quality	Best regime
KIVI 4-bit	4×	Lossless (.292)	Batch serving $\geq$ BS=16
KIVI 2-bit	8×	−6.2% (.273)	Max-batch, accept noise
TopK K=1024	1×†	Lossless (.292)	Single-req, long context
SnapKV 0.4	2.5×	+0.3% (.295)	Any; quality-neutral
MLA (B=1)	3.9×	−65%* needs retrain	After finetuning at $\geq$ 4K

† TopK does not reduce KV storage; * truncation mismatch may inflate gap.

V. Discussion

A. Deployment Recommendations

Table IV summarizes the operating point of each method.

KIVI 4-bit is the headline recommendation for high-throughput batch inference: it costs nothing in quality, extends the maximum serviceable batch size from 32 to 128, and multiplies peak throughput $3.1\times$ on a single H100. The only cost is TTFT latency (+18–66%), which matters for interactive applications.

SnapKV 0.4 is the best choice when quality must be preserved and per-step overhead must be zero. Its $2.5\times$ KV reduction adds no per-step decode cost because the compressed cache is built once after prefill.

TopK K=1024 is suited to single-request, long-context scenarios ($>32K$) where the paged variant (TopK-Flash) provides real TTFT and memory gains. At short context and low batch, scoring overhead dominates.

MLA shows the theoretical ceiling of latent projection: $3.9\times$ KV reduction and $1.3\times$ decode speedup are repeatable benefits even at B=1, but realizing this on a production-quality checkpoint requires finetuning with a longer context window and higher latent rank.

B. Kernel-Level Observations

INT4 GEMV inefficiency. `cuda_bmm_fA_qB_outer` performs per-GEMV INT4→FP16 dequantization. H100 tensor cores target INT8/FP16 GEMM; INT4 GEMV falls back to scalar or vector-INT4 paths and runs $\approx 2.5\times$ slower than native FP16 GEMV at BS=1. The bandwidth saving only exceeds this overhead when decode approaches the GEMM regime at BS $\geq$ 16.

KIVI residual buffer hot path. At short generation lengths ($< \text{residual_length} \times n_{\text{layers}}$), nearly all attention concentrates in the FP16 residual; KIVI provides minimal storage benefit. This is also why KIVI TTFT is $3\times$ slower: full-prefill quantization is paid unconditionally.

Triton fused scoring. `_topk_score_kernel` fuses $\mathbf{QK}^\top$ + softmax + head-sum into a single kernel launch, reducing Python-dispatch overhead 2– $3\times$ vs. sequential PyTorch ops. At 1K–4K sequence lengths the critical path remains HBM bandwidth, not compute.

C. Limitations

Single model architecture. All results are specific to Llama-2-7B with multi-head attention (32 heads, 32 KV heads). Models with grouped-query attention (Llama-3, Mistral) have fewer KV heads, changing the compression profile.

Small evaluation set. LongBench results are based on 20 examples per task. Some per-task deltas (e.g., qasper KIVI 2-bit −22%) may partially reflect small- n variance rather than a systematic effect, although the direction is consistent with the expected behavior.

MLA checkpoint mismatch. The available TransMLA checkpoint was calibrated at 256-token context without task finetuning. A properly trained MLA checkpoint would likely recover much of the 65% quality gap.

Hybrid scoring cost. The KIVI×TopK hybrid’s memory win (51%) comes with a decode penalty (−36%) because scoring must precede selection. Future work using a hierarchical or lazy-scoring approach could break this dependency.

VI. Conclusion

We conducted a controlled benchmark of five KV cache optimization methods across four compression families on Llama-2-7B. KIVI 4-bit provides lossless quality with $4\times$ KV compression and $1.93\times$ throughput at BS=32, making it our recommended default for batch-oriented serving. SnapKV marginally improves LongBench overall score with zero runtime overhead. TopK K=1024 matches base-line quality while gaining on noise-heavy retrieval tasks. MLA’s per-token latent representation yields consistent $3.9\times$ KV reduction and $1.3\times$ decode speedup at B=1, but requires a properly trained checkpoint for production quality. No single method dominates all axes; practitioners should select based on whether batch size, context length, or output fidelity is the primary constraint.

Future work should address: (i) batched decode for TopK to amortize the per-step scoring cost and achieve a KIVI-style crossover at BS $\geq$ 8; (ii) MLA retraining at 4K–8K context with rank 16–32 to preserve the $3.9\times$ compression gain while recovering QA quality; (iii) lazy or hierarchical scoring in the KIVI×TopK hybrid to decouple the memory win from the decode penalty; and (iv) cross-method composition (e.g., SnapKV eviction + KIVI quantization on the retained tokens) to stack savings from orthogonal axes.

Appendix

AI Use Disclosure

Per the HPML AI Use Policy. We used Claude (Anthropic) and GitHub Copilot for: debugging CUDA OOM errors, clarifying Triton kernel semantics, and polishing report prose. All reported experimental numbers were produced by running our own code on Modal H100 containers. All code was reviewed and validated by team members.

References

- [1] J. Jiang, P. Yang, R. Zhang, and F. Liu, “Towards efficient large language model serving: A survey on system-aware KV cache optimization,” TechRxiv, 2026.
- [2] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” arXiv:2307.09288, 2023.
- [3] Z. Liu et al., “KIVI: A tuning-free asymmetric 2-bit quantization for KV cache,” in Proc. ICML, 2024.
- [4] C. Hooper et al., “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in NeurIPS, 2024.
- [5] W. Wu et al., “TokenSelect: Efficient long-context inference via dynamic token-level KV cache selection,” in Proc. EMNLP, 2025.
- [6] P. Behnam et al., “RocketKV: Accelerating long-context LLM inference via two-stage KV cache compression,” in Proc. ICML, 2025.
- [7] F. Xu, T. Goyal, and E. Choi, “RefreshKV: Updating small KV cache during long-form generation,” in Proc. ACL, 2025.
- [8] Z. Zhang et al., “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in NeurIPS, 2023.
- [9] Y. Li et al., “SnapKV: LLM knows what you are looking for before generation,” in NeurIPS, 2024.
- [10] Y. Feng et al., “Ada-KV: Optimizing KV cache eviction by adaptive budget allocation,” in NeurIPS, 2025.
- [11] DeepSeek-AI, “DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model,” arXiv:2405.04434, 2024.
- [12] Y. He et al., “TransMLA: Multi-head latent attention is all you need,” arXiv:2502.07864, 2025.
- [13] C.-C. Chang et al., “xKV: Cross-layer SVD for KV-cache compression,” arXiv:2503.18893, 2025.
- [14] Y. Bai et al., “LongBench: A bilingual, multitask benchmark for long context understanding,” arXiv:2308.14508, 2023.